

Déploiement d'un site web multipage statique sur GitHub avec R Markdown

Cours Théorique
La SG-SERVICE
2025-12-29



Introduction

Aujourd'hui, **diffuser** du contenu scientifique, pédagogique ou analytique ne se limite plus à la production de **fichiers PDF** ou de **diapositives**. Que l'on soit **étudiant**, **enseignant**, **statisticien**, **data scientist** ou **chercheur**, il devient essentiel de savoir mettre ses contenus en ligne de manière **claire**, **accessible** et **professionnelle**. Cependant, le développement web classique (*HTML, CSS, JavaScript, serveurs, bases de données*) représente souvent une barrière technique importante, en particulier pour les profils orientés statistiques, données et recherche. C'est précisément pour répondre à ce besoin que **R Markdown**, associé à **GitHub Pages**, constitue une solution idéale :

- Le contenu est rédigé comme un **document R classique**;
- Le *texte*, le *code R* et les résultats sont intégrés dans un même fichier;
- Les *pages web HTML* sont générées **automatiquement**;
- Le site est publié **gratuitement** en ligne, sans gestion de serveur.



Objectif du cours

À l'issue de ce cours, toute personne disposant des bases du langage **R** sera capable de :

- ❶ Créer un **site web multipage**, composé de plusieurs pages liées entre elles ;
- ❷ Intégrer du **contenu dynamique** (graphiques, tableaux, résultats) ;
- ❸ Organiser la navigation du site à l'aide d'un **menu** ;
- ❹ Déployer le site sur **GitHub Pages** ;
- ❺ Mettre à jour le site de manière **simple** et **efficace**.

Aucune **connaissance** préalable en **développement web** n'est requise.



Principe général

Avant d'entrer dans la pratique, il est fondamental de comprendre le **fonctionnement global** du système. La solution repose sur quatre principes simples. Les deux premiers principes sont les suivants :

① *Une page du site correspond à un fichier .Rmd*

- Chaque page du site web est construite à partir d'un fichier **.Rmd**. Autrement dit : *une page web = un document R Markdown*.
- Ce document peut contenir simultanément : du texte (contenu pédagogique ou explicatif); du code R; des résultats produits automatiquement (graphiques, tableaux, indicateurs).

② *Un fichier central **_site.yml** décrit la structure du site*

- L'organisation globale du site est définie dans un fichier de configuration unique, nommé **_site.yml**.
- Ce fichier permet notamment de spécifier : les titres et sous-titres du site ; la structure du menu de navigation ; l'ordre et les liens entre les pages ; certains éléments de mise en forme et d'apparence.
- Il joue le rôle de **plan directeur** du site web.



Principe général

Les deux autres principes sont :

- ❶ ***R Markdown génère automatiquement*** les pages ***HTML***
 - Lors de la compilation, ***R Markdown*** transforme chaque fichier ***.Rmd*** en une page web au format ***.html***.
 - Cette opération est réalisée à l'aide d'une commande unique, sans ***intervention manuelle*** sur le code ***HTML***.
 - L'ensemble du site est ainsi généré de manière cohérente et reproductible.
- ❷ ***GitHub Pages héberge gratuitement le site***
 - Les fichiers ***.html*** générés sont ensuite hébergés par ***GitHub Pages***.
 - **Ce service permet** : une mise en ligne gratuite ; l'accès au site via une URL publique ; l'absence totale de gestion de serveur.
- ***Remarque importante :***
 - Le site est **statique du point de vue du serveur**, mais **dynamique au moment de la génération** : à chaque mise à jour, les calculs, graphiques et tableaux sont automatiquement recalculés et intégrés aux pages web.



→ Outils nécessaires

- Avant de commencer, il est indispensable de disposer des **éléments** suivants :

- ① Une version récente du logiciel **R**;
- ② L'environnement de développement : **RStudio** ;
- ③ un compte **GitHub** (*gratuit*);
- ④ une connexion à **Internet**.

- **Packages R indispensables**

- ① Dans **RStudio**, les packages suivants doivent être installés une fois pour toutes :
- ② `install.packages(c("rmarkdown", "knitr", "ggplot2"))`
- ③ Ces packages jouent un rôle central dans la construction du *site web* :
 - **rmarkdown** permet la génération des pages du site ;
 - **knitr** assure l'exécution du *code R* et l'*intégration des résultats* ;
 - **ggplot2** est utilisé pour la production de *graphiques*.



Création du projet : poser les bases correctement

→ Création du dossier de travail

Dans **RStudio**, procéder comme suit :

- **File** → **New Project** → **New Directory** → **Empty Project** ;
- Nommer le projet, par exemple : `mon_site_rmarkdown`.

Le dossier ainsi créé contiendra l'intégralité des fichiers nécessaires à la construction et au déploiement du `site web`.

→ Structure minimale du site (fondamentale)

L'arborescence minimale recommandée est la suivante :

- `mon_site_rmarkdown/`
 - ① `_site.yml`
 - ② `index.Rmd`
 - ③ `cours1.Rmd`
 - ④ `cours2.Rmd`
 - ⑤ `styles.css`
 - ⑥ `docs/`



Création du projet : poser les bases correctement

→ Rôle de chaque élément

- `_site.yml` : Fichier de configuration générale du site ;
- `index.Rmd` : Page d'accueil du site ;
- `coursX.Rmd` : Pages de contenu (cours, analyses, projets) ;
- `styles.css` : Personnalisation de l'apparence visuelle du site ;
- `docs/` : Dossier de sortie utilisé par GitHub Pages.

● **Attention :**

- Il ne faut **jamais écrire directement** dans le dossier `docs/`. Ce dossier est généré automatiquement lors de la compilation du site.

→ Le fichier `_site.yml` : le cerveau du site

Le fichier `_site.yml` constitue le cœur du site. Il permet de définir :

- Le nom du site ;
- Le menu de navigation ;
- L'apparence graphique et le dossier de sortie.



Exemple complet du fichier `_site.yml`

- **name** : *"mon_site_cours"*
- **output_dir** : "docs"
- **navbar** :
 - **title** : "Nos Cours"
 - **left** :
 - - text : "Accueil"
 - **href** : index.html
 - - text : "Liste des cours"
 - **menu** :
 - - text : "cours 1"
 - **href** : cours1.html
 - - text : "cours 2"
 - **href** : cours2.htm
 - - text : "cours 3"
 - **href** : cours3.html
 - - text : "cours 4"
 - **href** : cours4.html
 - - text : "cours 5"
 - **href** : cours5.html
- **output** :
 - **html_document** :
 - **theme** : flatly
 - **highlight** : tango
 - **css** : styles.css

À retenir absolument

- chaque **lien** du menu pointe vers un fichier `.html`
- chaque fichier `.html` correspond à un fichier `.Rmd`
- **output_dir** :
"docs" est indispensable pour **GitHub Pages**



Création de la page d'accueil

La page d'accueil constitue le point d'entrée principal du *site*. Elle permet de présenter le *contenu global* du cours et d'orienter l'utilisateur vers les différentes sections via le menu de navigation.

- **Étape 1 : Création du fichier**

- ❶ Créer un fichier nommé `index.Rmd`.
- ❷ Ce fichier correspondra à la page d'accueil du site.

- **Étape 2 : Entête YAML minimal**

- L'entête YAML définit les paramètres essentiels de la page (titre et format de sortie).
- **Exemple :**
 - ❶ `title: "Accueil"`
 - ❷ `output: html_document`

- **Étape 3 : Contenu de la page**

- Le *contenu principal* de la page est rédigé en **Markdown**.

À retenir : La page `index.Rmd` est automatiquement reliée au menu de navigation défini dans le fichier `_site.yml`. Aucune configuration supplémentaire n'est nécessaire.



Chaque **cours du site** correspond à un fichier `.Rmd` indépendant. Cela permet de structurer le contenu pédagogique de manière modulaire (**un fichier = un cours**).

- **Exemple : création d'un fichier de cours**

- Créer un fichier nommé `cours1.Rmd`.
 - **En-tête YAML du cours** : L'en-tête YAML définit le titre du cours et le format de sortie HTML.
 - **Contenu pédagogique du cours** : Le contenu est rédigé en Markdown et peut inclure du texte, des équations, des tableaux et du code R.
 - **Ajout de contenu dynamique avec R** : R Markdown permet d'intégrer du code R directement dans le document. Le code est exécuté automatiquement lors de la génération du site.

Génération complète du site web

Une fois l'ensemble des fichiers *.Rmd* correctement configurés, la génération du *site web* se fait à l'aide d'une *seule commande R*.

- **Commande de génération**

- Dans la **console** ou un **chunk R**, exécuter: *rmarkdown::render_site()*.

- **Rôle de la commande *render_site()***

- Cette commande automatise l'intégralité du processus de déploiement du site :

- 1 Elle exécute tous les fichiers *.Rmd* présents dans le projet ;
- 2 Elle compile chaque fichier en une page *.html* ;
- 3 Elle organise l'ensemble des pages générées dans le dossier *docs/*, conformément à la configuration du fichier *_site.yml*.

- Le dossier *docs/* constitue le point d'entrée du site web et sera utilisé par *GitHub Pages* pour la publication en ligne.

- **Vérification locale du site**

- 1 Avant toute mise en ligne, il est indispensable de vérifier le *rendu local du site*.
- 2 Ouvrir le fichier suivant dans un navigateur web: *docs/index.html*.



→ Points de contrôle recommandés

Lors de cette *vérification*, s'assurer notamment que :

- La *navigation* entre les pages fonctionne correctement ;
- Les *menus* sont bien affichés et cohérents avec la structure du site ;
- Les *graphiques* et *tableaux* sont correctement rendus ;
- Le *contenu* est homogène et conforme aux objectifs pédagogiques du site.

Cette étape permet de *corriger d'éventuelles erreurs* avant le déploiement définitif sur *GitHub Pages*.

Mise en ligne du site avec GitHub Pages

● **Création du dépôt GitHub**

- La première étape consiste à créer un dépôt distant qui hébergera le site.
 - 1 Se rendre sur la plateforme GitHub ;
 - 2 **Créer** un nouveau dépôt (ex : `mon_site`) ;
 - 3 Ne rien **ajouter** automatiquement lors de la création (pas de README).
- Cette configuration permet de lier proprement un projet local existant au **dépôt distant**.

● **Lier le projet local au dépôt distant**

- Depuis le **dossier racine** du projet **R Markdown**, exécuter les commandes suivantes dans un terminal :
 - 1 `git init`
 - 2 `git add .`
 - 3 `git commit -m "Création du site R Markdown"`
 - 4 `git branch -M main`
 - 5 `git remote add origin https://github.com/USERNAME/mon\_site.git`
 - 6 `git push -u origin main`
- Ces commandes permettent :
 - 1 D'**initialiser** le dépôt *Git local*, D'**enregistrer** l'ensemble des fichiers du site, De **relier** le projet local au dépôt **GitHub** ;
 - 2 De publier le contenu sur la branche principale `main`.



- **Lier le projet local au dépôt distant**

- Une fois le dépôt correctement alimenté, l'activation de **GitHub Pages** s'effectue directement depuis l'interface *GitHub* :

- ① **Accéder** à l'onglet Settings du dépôt ;
- ② **Sélectionner** Pages dans le menu latéral ;
- ③ **Choisir** Deploy from a branch comme source ;
- ④ **Sélectionner** la branche main ;
- ⑤ **Définir** le dossier /docs comme répertoire de publication ;
- ⑥ **Cliquer** sur Save.

- **Accès au site en ligne**

- Après quelques **secondes** ou **minutes**, le site est automatiquement mis en ligne et accessible à l'adresse suivante : ***https://USERNAME.github.io/mon_site/***. Cette URL constitue le point d'accès public au site hébergé sur *GitHub Pages*.

Téléversement des fichiers directement depuis GitHub

Cette méthode permet de publier un site web *sans utiliser Git en ligne de commande*. Toutes les étapes sont réalisées directement depuis l'interface graphique de **GitHub**.

- **Étape 1 : Connexion à GitHub**

- Ouvrir un navigateur **web** ;
- Se rendre sur le site `https://github.com` ;
- Se connecter à son **compte GitHub** à l'aide de ses identifiants (*adresse e-mail et mot de passe*).

- **Étape 2 : Création du dépôt**

- Une fois connecté : cliquer sur le bouton **New** ou **New repository** ;
- Saisir le nom du **dépôt** (exemple : `mon_site_html`) ;
- Laisser le **dépôt** en mode **Public** ;
- **Ne rien cocher** :
 - ❶ Ne pas **ajouter** de **README** ;
 - ❷ Ne pas **ajouter** de **licence** ;
 - ❸ Ne pas **ajouter** de `.gitignore`.
- Cliquer sur **Create repository**.



Téléversement des fichiers directement depuis GitHub

● **Étape 3 : Accès à l'interface de téléversement**

- Après la création du dépôt :
 - ➊ Rester sur la page principale du dépôt ;
 - ➋ Cliquer sur le bouton **Add file** ;
 - ➌ Sélectionner **Upload files**.

● **Étape 4 : Téléversement des fichiers HTML**

- Dans la page de téléversement :
 - ➊ **Glisser-déposer** les fichiers **.html** depuis l'ordinateur ;
 - ➋ Vérifier que le fichier **index.html** est bien présent (il correspond à la page d'accueil du site) ;
 - ➌ **Glisser-déposer** également les dossiers nécessaires (**images/**, **css/**, etc.).

- **Important** : les fichiers **.html** doivent être placés à la **racine du dépôt** (et non dans un sous-dossier).

● **Étape 5 : Validation du téléversement**

- En bas de la page :
 - ➊ Laisser le message de **commit** par défaut (ou saisir un message explicite, par exemple **Ajout des fichiers HTML**) ;
 - ➋ Cliquer sur **Commit changes**.

- **Les fichiers sont alors enregistrés dans le dépôt GitHub.**



Téléversement des fichiers directement depuis GitHub

● **Étape 6 : Activation de GitHub Pages**

- Toujours depuis le dépôt **GitHub** :

- ❶ Cliquer sur l'onglet **Settings** ;
- ❷ Dans le menu latéral, sélectionner **Pages** ;
- ❸ Dans la section **Source**, choisir **Deploy from a branch** ;
- ❹ Sélectionner la branche **main** ;
- ❺ Choisir le dossier / (root) ;
- ❻ Cliquer sur **Save**.

● **Étape 7 : Accès au site en ligne;**

- Après quelques secondes, GitHub affiche l'adresse du site web :

- ❶ `https://USERNAME.github.io/mon_site_html/`;
- ❷ Cette URL permet d'accéder directement au site publié via **GitHub Pages**.

● **Point de vigilance**

- ❶ Toute modification d'un fichier nécessite un **nouveau téléversement** ;
- ❷ L'absence du fichier `index.html` empêche l'affichage du site ;
- ❸ Cette méthode est **recommandée uniquement pour des sites simples**.



- **Bonnes pratiques, limites en utilisant Github et R Markdown**

- **Bonnes pratiques essentielles**

- ① Un fichier `.Rmd` par page, Titres clairs et hiérarchisés.
- ② Menu de navigation simple, Peu de dépendances externes.
- ③ Code commenté, Données reproductibles.

- **Cas d'usage typiques**

- ① Site de **cours universitaire**
- ② Supports de **travaux dirigés (TD)** et de **travaux pratiques (TP)**
- ③ **Portfolio** en *data science* ou en *statistique*;
- ④ Présentation de **projets académiques** ou **professionnels**
- ⑤ Rapports **scientifiques reproductibles**

- **Limites à connaître**

- ① Absence de calcul côté utilisateur (**site statique**)
- ② Pas de gestion de base de données
- ③ Interactivité limitée aux fonctionnalités HTML/CSS/JavaScript simples

- **Extensions et évolutions possibles**

- ① **Shiny** : pour ajouter de l'interactivité et des calculs dynamiques
- ② **JavaScript** : pour enrichir l'expérience utilisateur
- ③ **Quarto**: pour une gestion moderne et unifiée des documents et sites



CAS PRATIQUE : Notre site web déployé sur GitHub

Ce cas pratique présente *le déploiement d'un site web pédagogique statique* dédié aux statistiques, à l'économétrie et à la data science, entièrement réalisé avec R Markdown et publié via **GitHub Pages**, sans serveur ni développement web complexe.

Le site propose une navigation claire et un menu déroulant donnant accès à plusieurs cours (*statistiques descriptives, régression logistique multinomiale, tests de comparaison de moyennes, théorie des sondages, modèles de Poisson et de Cox*), ainsi qu'une présentation des activités *de La SG-SERVICE*. Il illustre concrètement comment structurer des contenus pédagogiques multipages à partir de fichiers *.Rmd*, assurer leur reproductibilité et les diffuser publiquement en ligne.

Le site est accessible à l'adresse suivante : <https://lasg-service.github.io/site-web-statistiques-eco-datascience/>.



MERCI POUR VOTRE ATTENTION !!!

